大作业 2of2 实验指导书：

简易编译器 QCompiler 实现及验证

本实验完整复现现代编译器词法分析→语法分析→AST 生成→中间代码生成→目标代码生成→代码优化全流程，基于自定义 QL 语言、QTAC 中间表示，最终输出 ARM 汇编并在鲲鹏平台验证运行，覆盖编译原理核心技术点。

一、实验基本信息

实验名称：简易编译器 QCompiler 实现及验证

实验学时：[根据教学安排填写]

适用对象：计算机相关专业编译原理课程

源语言：自有 QL 语言

中间语言：自有 QAST 抽象语法树、自有 QTAC 语言

目标架构：ARM64（华为鲲鹏）

二、实验目的

本实验通过理论与实践结合，掌握编译程序完整实现要点，包括词法分析、语法分析、抽象语法树、符号表、中间表示、运行时环境、目标代码生成、优化。

本实验将编译原理与编译器工程实现相衔接，通过理论落地来加强理论知识的融会贯通和抽象能力提升。

运用 AI 辅助完成 QL 语言简易编译器的设计和代码实现，提升工程能力。

编程实现编译器全流程逻辑，使用 qsort 测试程序验证各阶段输出。

在华为鲲鹏 ARM 环境完成汇编、链接、运行，验证目标代码正确性。

三、实验环境

1. QCompiler 开发环境

自备、不做任何限制。

2. AI 工具

自备、不做任何限制。

3. 目标程序的运行平台

硬件平台：华为鲲鹏服务器 / 开发板（ARM64 架构，推荐 Kunpeng 920）

操作系统：openEuler（推荐）、Ubuntu ARM64、CentOS ARM64

开发工具链如下：

编译器：aarch64-linux-gnu-gcc / LLVM/Clang

汇编器：as (GNU Assembler)

链接器：ld (GNU Linker)

调试工具：gdb

文本 / 开发工具：Vim、VSCode、IDEA 等

4. 实验素材（另附）

QL 源语言文法与语义定义文件

QAST 抽象语法树结点定义文件

QTAC 中间语言指令集定义文件

Q2ARM 汇编指令模板库

测试用例：qsort 快速排序 QL 源码

四、实验原理

1. 编译器整体架构

QL 源程序

↓【词法分析】

Token 流

↓【语法分析 LL(1)/SLR(1)】

QAST 抽象语法树

↓【中间代码生成】

QTAC 三地址代码、符号表

↓【σ-DFA + 代码生成】

ARM64 汇编代码

↓【代码优化】

优化后 ARM64 汇编

↓【汇编+链接+运行】

鲲鹏平台可执行程序

2. **核心技术**

词法分析：正则表达式、状态机、Token 切分与类型标记

语法分析：LL (1) 递归下降、SLR (1) 表驱动分析

语义及中间表示：QAST 结构设计、QTAC 三地址码

代码生成：σ-DFA 状态机、Q2ARM 模板映射

代码优化：局部优化 + 寄存器分配

系统运行：ARM64 汇编、静态链接、鲲鹏环境执行

五、实验内容与步骤

任务步骤 1：词法分析器实现（ABC 公用）

定义 QL 语言关键字、标识符、常量、运算符、界符

构建词法规则，实现 DFA 或正则匹配

输入 QL 程序，输出：(类型, 值) 格式 Token 序列

测试用例：qsort.ql → tokens.txt

任务步骤 2A：LL (1) 递归下降分析（仅 A）

对 QL 文法提取左公因子、消除左递归

构造 SELECT 集，验证 LL (1) 合法性

为每个非终结符编写递归下降分析函数

输入 Token 流，构建并输出 QAST

任务步骤 2B：SLR(1) 分析框架（仅 B）

增广文法，构造 LR(0) 项目集规范族

构造 SLR(1) 分析表（ACTION/GOTO）

实现表驱动移进归约分析程序

生成 QAST 抽象语法树

任务 2AB：AST 遍历生成 QTAC

设计 QAST 结点结构（表达式、语句、函数、控制流）

实现前序 / 后序遍历，执行语义动作

输出标准 QTAC 三地址代码和符号表

任务步骤 2C：SLR(1) 制导翻译（仅 C）

增广文法，构造 LR(0) 项目集规范族

构造 SLR(1) 分析表（ACTION/GOTO）

实现表驱动移进归约分析框架

在 SLR(1) 分析过程中嵌入语义规则

移进归约时直接生成 QTAC 和符号表

跳过 QAST 阶段，实现源语言到中间代码直接翻译

任务步骤 3：QTAC → ARM64 汇编生成（ABC 公用）

来利用 QTAC 到 ARM 的指令模板 Q2ARM

构建 σ-DFA 指导寄存器选择与指令编排

实现代码生成器，输出 .s 汇编文件

处理函数调用、内存访问、控制流跳转

任务步骤4：ARM汇编代码优化（任意步骤任选）

复制传播优化

无用指令（死代码）删除

公共子表达式消除

叶子函数优化（省去栈帧）

图着色寄存器分配

输出优化后的ARM汇编

任务步骤5：整体测试与鲲鹏验证（ABC公用）

使用qsort测试全编译器链路：

QL → Token → AST（仅 AB） → QTAC → ARM → opt_ARM

在鲲鹏平台执行汇编与链接：

bash

运行

as -o qsort.o qsort.s

ld -o qsort qsort.o

运行程序，验证排序功能正确性

记录各阶段输出文件与运行结果

六、实验输出物

词法分析器源码 + Token输出文件

LL (1) 分析器源码 + QAST文件（仅 A）

SLR (1) 分析器源码 + QAST文件（仅 B）

遍历 QAST 的语义分析及 QTAC 代码生成源码（仅 AB）

语法制导翻译源码（含语义分析及 QTAC 代码生成）（仅 C）

QTAC 中间代码及符号表文件

σ-DFA 转换 QTAC 为 ARM 代码的程序源码

未优化 ARM 汇编文件

优化后 ARM 汇编文件及各优化遍源码（有、无均可）

鲲鹏平台可执行文件（若有手工调整需指出）＋运行截图

实验报告（含对各任务步骤完成过程的简要描述和输出结果的简要分析、AI 的使用方式及效果的简要描述、值得说明的问题解决及问题遗留、收获及展望。所有源码压缩作为报告附件。）

七、实验注意事项

实验任务分为 A、B 和 C 三种，任选一种完成即可。

优化子任务为任选，包括任务步骤 5 中给出的几个优化遍。如果没有选寄存器分配则需要手工替换 ARM 代码中的虚拟机寄存器为物理寄存器。

对 QCompiler 的实现语言不做限制。

对 QL 文法做必要的修剪或定义额外的冲突消解规则以满足 LL(1)/SLR(1)分析的条件。

Token 定义必须统一，避免语法分析出错。

QAST 结点设计要覆盖所有语法结构。

ARM64 调用约定、寄存器使用必须规范。

鲲鹏平台必须使用原生 ARM 工具链汇编链接。

优化必须保证语义不变，不能破坏程序功能。

八、实验评分标准

由老师根据实验报告及附件中提供的完成情况进行综合打分。

实验报告及附件提交至邮箱 zyl9910@xjtu.edu.cn。邮件标题和附件名称保持一样，为：*姓名*+QCompiler+*X*，其中 *X* 为 A、B 或 C。

注意，zyl9910 中第 3 个字符是字母 L 的小写